Programming Worlds

PROGRAMMING WORLDS ACADEMY

Git & GitHub 101

From Zero to Your First Pull Request

BEYOND CODE. BEYOND LIMITS.

12

MODULE 12

Final Project: Portfolio Website

Use everything you learned to build, review and publish a portfolio website with Git and GitHub.

MODULE 12 · LESSON 12.1

Plan and start the project

التخطيط وبدء المشروع

Learning objectives

- Read the project brief.
- Create the repository.
- Plan branches and tasks.

Prerequisites

Modules 1–11.

Introduction

You will build a small portfolio website and manage it like a real team project.

Explanation

Brief. A one-page site with four sections: Home, About, Projects, Contact. Plain HTML and CSS; no frameworks.

Repository. Create `portfolio-website` on GitHub (public, with no README), then run the commands in Figure 1. Use `git init -b main` so the first branch is called main.

Add a starter structure: `index.html`, `style.css`, `README.md`, `.gitignore`.

Plan. Write tasks as branches: `feature/home`, `feature/about`, `feature/projects-section`, `feature/contact`, `docs/readme`.

Rules. Never commit to main after the first push; every change goes through a pull request; commit messages are specific.

You can use GitHub **Issues** to list tasks and write `Closes #1` in a PR to close one.

What the website needs. A navigation bar, a hero section with a heading and a button, an About section, a Projects section with cards, a Contact section, and a footer. Make it responsive with a viewport meta tag and a flexible grid. The folder `final-project/starter` has the empty skeleton and `final-project/solution` has one complete example to compare with afterwards.

Real-world analogy

> You are the architect and the site manager at once: first the plan, then the building.

Diagram



Figure 1 · Illustration: starting the repository

Commands

`git init -b main`
`git remote add origin URL`
`git push -u origin main`

Figure 1 shows the setup.

Common mistakes and solutions

Creating the repo with a README and then pushing a different history.	Create it empty, or clone it instead.
Starting without a plan.	List branches first.
Working on main.	Branch for every task.

Practical exercise

> Create the repository and push the skeleton.

Expected result

GitHub shows your files and a single commit on main.

Knowledge check

1. How many sections does the brief require?

- ○ Ten
- ○ Four
- ○ One

2. Where do changes go after the first push?

- ○ Through branches and PRs
- ○ Directly to main
- ○ Nowhere

3. Which command names the first branch main?

- ○ git add main
- ○ git init -b main
- ○ git clone main

Answers

- Four — Home, About, Projects, Contact.
- Through branches and PRs — Team workflow.
- git init -b main — It avoids later renaming.

Summary

Create an empty repo, push a skeleton, and plan tasks as branches.

Key vocabulary

Skeleton	Minimal starting structure
Issue	Task or bug tracker item
Brief	Project requirements

Next lesson

Lesson 12.2 — Build with branches and pull requests.

MODULE 12 · LESSON 12.2

Build with branches and pull requests

البناء بالفروع وطلبات الدمج

Learning objectives

- Build each section on its own branch.
- Open and merge PRs.
- Keep history clean.

Prerequisites

Lesson 12.1.

Introduction

Repeat one loop for each task: branch, build, commit, push, pull request, merge, pull.

Explanation

The loop (Figure 1): `git switch main`, `git pull`, `git switch -c feature/projects-section`.

Edit the files in VS Code and look at the page in your browser.

Commit in small steps, such as “Add projects heading” and “Style project cards”.

Push, open a pull request with a clear description, review **Files changed**, and merge.

Pull main and delete the branch locally.

Do this for at least **three** branches and **two merged pull requests**. Deliberately create one conflict in a safe branch and resolve it if you want extra practice.

Add a `.gitignore` and a useful `README.md` (Module 11).

Real-world analogy

> Each branch is a small renovation: you finish one room before starting the next.

Diagram



Figure 1 · Illustration: one task loop

Commands

`git switch -c feature/NAME`

`git add .` or specific files

`git commit -m "message"`

`git push -u origin feature/NAME`

Common mistakes and solutions

One giant branch for the whole site.	Split by section.
Merging with failing page.	Open the page and test first.
Forgetting to pull before the next branch.	Always pull main.

Practical exercise

> Complete the loop for Home, Projects and Contact sections.

Expected result

GitHub shows at least two merged pull requests and 8 or more commits.

Knowledge check

1. What starts each task?

- ○ git clone
- ○ git switch main and git pull
- ○ git reset –hard

2. How many merged PRs are required?

- ○ At least two
- ○ None
- ○ Fifty

3. What do you test before merge?

- ○ Only the README
- ○ The page in the browser
- ○ Nothing

Answers

- git switch main and git pull — Branch from fresh main.
- At least two — See rubric.
- The page in the browser — Make sure it works.

Summary

Branch per task, small commits, PRs, merge, then pull main.

Key vocabulary

Loop	Repeated workflow
Merged PR	Pull request combined into main
Clean history	Readable commits

Next lesson

Lesson 12.3 — Publish and submit.

MODULE 12 · LESSON 12.3

Publish with GitHub Pages and submit

النشر والتسليم

Learning objectives

- Publish a static site.
- Verify the live URL.
- Submit the project.

Prerequisites

Lesson 12.2.

Introduction

GitHub Pages hosts your static site for free from a repository.

Explanation

On GitHub open the repository, then **Settings → Pages**.

Under **Build and deployment**, set **Source** to **Deploy from a branch**, choose **main** and **/ (root)**, and click **Save**.

Wait one or two minutes and refresh. GitHub shows the address: `https://YOUR-USERNAME.github.io/portfolio-website/` .

Open the address and check all links and the CSS. Paths must be relative, such as `style.css` .

Make a fix by branch and PR; after you merge, the site updates automatically within minutes.

Submit: repository URL, live URL, and a short note listing your branches and PRs. Check the rubric before you submit.

GitHub Pages on a free account requires a public repository. Check current GitHub plan limits if you need a private one.

Real-world analogy

> Publishing is opening the shop doors: the work you did behind the scenes is now visible.

Diagram

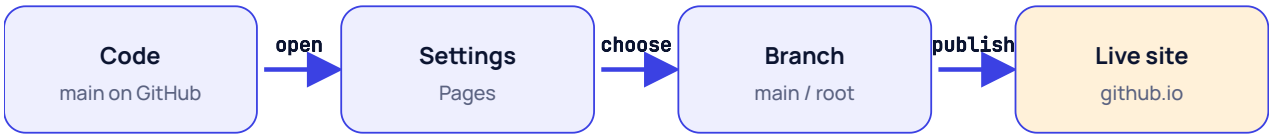


Figure 1 · Publishing path

Commands

No new Git command. Pages is configured on GitHub.

Figure 1 shows the publishing path.

Common mistakes and solutions

Page shows 404.	Wait a few minutes and check branch and folder settings, and that index.html is in the root.
Broken CSS.	Use relative paths and exact letter case.
Private repository on a free plan.	Make it public for Pages.

Practical exercise

> Publish the site and send the two URLs.

Expected result

The live URL shows your portfolio.

Knowledge check

1. Where do you enable Pages?

- ○ VS Code Extensions
- ○ Settings → Pages
- ○ Git Bash

2. Which file must be in the root?

- ○ index.html
- ○ notes.txt
- ○ .env

3. What do you submit?

- ○ Nothing
- ○ Repo URL and live URL
- ○ Only a screenshot

Answers

- Settings → Pages — It is a repository setting.
- index.html — It is the home page.
- Repo URL and live URL — Plus a short note.

Summary

Enable Pages from main/root, verify the live URL, and submit both URLs.

Key vocabulary

GitHub Pages	Free static hosting from a repository
Live URL	Public address of the site
Static site	HTML/CSS/JS files without a server

Next lesson

> **You finished the course.** Check the rubric and celebrate.

MODULE 12

Capstone lab · Portfolio website with Git and GitHub

Build the project from Lessons 12.1–12.3. Starter files are in `final-project/starter`; a sample solution is in `final-project/solution`.

1. Create the repository and push the skeleton.
2. Build Home, About, Projects and Contact on separate branches.
3. Open and merge at least two pull requests.
4. Add README.md and .gitignore.
5. Publish with GitHub Pages.
6. Submit both URLs.

Grading rubric

Criterion	Points	Full marks when…
Repository and structure	15	Clear names, index.html, style.css, README, .gitignore
Commit history	20	8+ commits with specific messages
Branches and pull requests	25	3+ branches, 2+ merged PRs with descriptions
Website content	20	Home, About, Projects, Contact work and look tidy
GitHub Pages	10	Live URL opens with working CSS
README and safety	10	README explains the project; no secrets committed

Total 100 · pass mark 70.

Module 12 quiz (pass mark: 70%)

1. What do you submit?

- ○ Only your password
- ○ Nothing
- ○ Repository URL and live URL

2. Which GitHub setting publishes the site?

- ○ Profile → Photo
- ○ Settings → Pages
- ○ Settings → Billing

3. Where should work be done?

- ○ On branches, merged by PR
- ○ Only on main
- ○ Outside Git

4. A 404 on the live URL — first check?

- ○ Delete the repo
- ○ Branch, folder and index.html in root
- ○ Reinstall Windows

5. What must never be committed?

- ○ index.html
- ○ README
- ○ Secrets and tokens

Answers

- Repository URL and live URL — Plus a short note.
- Settings → Pages — Choose main / root.
- On branches, merged by PR — Team workflow.
- Branch, folder and index.html in root — Also wait a few minutes.
- Secrets and tokens — Keep them out.